

You are here: [Home](#) | [Enterprise Java](#) | Implement Bootstrap Pagination With Spring Data And Thymeleaf

About Yuan Ji



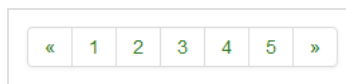
Implement Bootstrap Pagination With Spring Data And Thymeleaf

by Yuan Ji on March 5th, 2013 | Filed in: [Enterprise Java](#) Tags: [Spring](#), [Spring Data](#), [Thymeleaf](#)

Twitter Bootstrap has a very nice pagination UI, and here I will show you how to implement it with [Spring Data Web pagination](#) function and [Thymeleaf](#) conditional evaluation features.

Standard Pagination in Bootstrap

Simple pagination inspired by Rdio, great for apps and search results. The large block is hard to miss, easily scalable, and provides large click areas.



The original source code to display pagination from Bootstrap document is very simple:

```
01 <div class='pagination'>
02   <ul>
03     <li><a href='#'>Prev</a></li>
04     <li><a href='#'>1</a></li>
05     <li><a href='#'>2</a></li>
06     <li><a href='#'>3</a></li>
07     <li><a href='#'>4</a></li>
08     <li><a href='#'>5</a></li>
09     <li><a href='#'>Next</a></li>
10   </ul>
11 </div>
```

You can see this is just a mock up code, and to make it display page number dynamically with correct hyperlink URL, I need to do many changes to my existing code. So let's start from bottom up, change domain layer first, then application service layer, presentation layer. Finally the configuration to glue them together.

Domain Layer Changes

The only change in domain layer is at `BlogPostRepository`. Before it has a method to retrieve a list of published `BlogPost` sorted by `publishedTime`:

```
1 public interface BlogPostRepository extends MongoRepository<BlogPost, String>{
2   ...
3   List<BlogPost> findByPublishedIsTrueOrderByPublishedTimeDesc();
4   ...
5 }
```

Now we need to get paginated result list. With [Spring Data Page](#), we will return `Page<BlogPost>` instead of `List<BlogPost>`, and pass `Pageable` parameter:

```
1 public interface BlogPostRepository extends MongoRepository<BlogPost, String>{
2   ...
3   Page<BlogPost> findByPublishedIsTrueOrderByPublishedTimeDesc(Pageable pageable);
4   ...
5 }
```

Application Service Layer Changes:

The application service layer change is also very simple, just using new function from `BlogPostRepository`:

BlogService interface



Carrer Opportunities

Java developer – Cognizz
FULL-TIME) February 13

Java Developer – ACENT
FULL-TIME) February 13

Java Developer – Informa
Lansing, MI (FULL-TIME

Java Developer – MacNei
Sunrise, FL (FULL-TIME

Java Developers – Web C
States (FULL-TIME) Feb

Join Us

487,813 Feb u

Do you have a blog with un
content? Then, check out c
program. You can also be a
Code Geeks

Tags

Akka Android Tutorial Ap
Apache Maven Apacl
Big Data Books Cache Clc
Design Patterns Eclip
Google GWT Java 7 Java I
JAXB JBoss JBoss Hil
JSON JUnit JVM Log
NoSQL Oracle GlassF
Performance and Scalabi
Project Management RES
Security Sprin
Spring Security Te

```

1 public interface BlogService {
2     ...
3     Page<BlogPost> getAllPublishedPosts(Pageable pageable);
4     ...
5 }

```

BlogServiceImpl class

```

01 public class BlogServiceImpl implements BlogService {
02     ...
03     private final BlogPostRepository blogPostRepository;
04     ...
05     @Override
06     public Page<BlogPost> getAllPublishedPosts(Pageable pageable) {
07         Page<BlogPost> blogList =
08         blogPostRepository.findByPublishedIsTrueOrderByPublishedTimeDesc(pageable);
09         return blogList;
10     }
11     ...
12 }

```

Presentation Layer Changes:

Spring Data [Page](#) interface has many nice functions to get current page number, get total pages, etc. But it's still lack of ways to let me only display partial page range of total pagination. So I created an adapter class to wrap Spring Data Page interface with additional features.

```

01 public class PageWrapper<T> {
02     public static final int MAX_PAGE_ITEM_DISPLAY = 5;
03     private Page<T> page;
04     private List<PageItem> items;
05     private int currentNumber;
06     private String url;
07
08     public String getUrl() {
09         return url;
10     }
11
12     public void setUrl(String url) {
13         this.url = url;
14     }
15
16     public PageWrapper(Page<T> page, String url){
17         this.page = page;
18         this.url = url;
19         items = new ArrayList<PageItem>();
20
21         currentNumber = page.getNumber() + 1; //start from 1 to match page.page
22
23         int start, size;
24         if (page.getTotalPages() <= MAX_PAGE_ITEM_DISPLAY){
25             start = 1;
26             size = page.getTotalPages();
27         } else {
28             if (currentNumber <= MAX_PAGE_ITEM_DISPLAY - MAX_PAGE_ITEM_DISPLAY/2){
29                 start = 1;
30                 size = MAX_PAGE_ITEM_DISPLAY;
31             } else if (currentNumber >= page.getTotalPages() - MAX_PAGE_ITEM_DISPLAY/2){
32                 start = page.getTotalPages() - MAX_PAGE_ITEM_DISPLAY + 1;
33                 size = MAX_PAGE_ITEM_DISPLAY;
34             } else {
35                 start = currentNumber - MAX_PAGE_ITEM_DISPLAY/2;
36                 size = MAX_PAGE_ITEM_DISPLAY;
37             }
38         }
39
40         for (int i = 0; i<size; i++){
41             items.add(new PageItem(start+i, (start+i)==currentNumber));
42         }
43     }
44
45     public List<PageItem> getItems(){
46         return items;
47     }
48
49     public int getNumber(){
50         return currentNumber;
51     }
52
53     public List<T> getContent(){
54         return page.getContent();
55     }
56
57     public int getSize(){
58         return page.getSize();
59     }
60
61     public int getTotalPages(){
62         return page.getTotalPages();
63     }
64
65     public boolean isFirstPage(){
66         return page.isFirstPage();
67     }
68
69     public boolean isLastPage(){
70         return page.isLastPage();
71     }
72 }

```

```

73 public boolean isHasPreviousPage(){
74     return page.hasPreviousPage();
75 }
76
77 public boolean isHasNextPage(){
78     return page.hasNextPage();
79 }
80
81 public class PageItem {
82     private int number;
83     private boolean current;
84     public PageItem(int number, boolean current){
85         this.number = number;
86         this.current = current;
87     }
88
89     public int getNumber(){
90         return this.number;
91     }
92
93     public boolean isCurrent(){
94         return this.current;
95     }
96 }
97 }

```

With this `PageWrapper`, we can wrap `Page<BlogPost>` returned from `BlogService` and put it into SpringMVC UI model. See the controller code for the blog page:

```

01 @Controller
02 public class BlogController
03 ...
04 @RequestMapping(value = '/blog', method = RequestMethod.GET)
05 public String blog(Model uiModel, Pageable pageable) {
06     PageWrapper<BlogPost> page = new PageWrapper<BlogPost>
07         (blogService.getAllPublishedPosts(pageable), '/blog');
08     uiModel.addAttribute('page', page);
09     return 'blog';
10 }
11 ...
12 }

```

The `Pageable` is passed in from `PageableArgumentResolver`, which I will explain later. Another trick is I also pass the view URL to `PageWrapper`, and it can be used to construct `Thymeleaf` hyperlinks in pagination bar.

Since my `PageWrapper` is very generic, I created an html fragment for pagination bar, so I can use it for anywhere in my application pages when pagination needed. This fragment html uses `Thymeleaf` `th:if` to dynamically switch between static text or hyperlink based on if the link is disabled or not. And it uses `th:href` to construct URL with correct page number and page size.

```

01 <!-- Pagination Bar -->
02 <div th:fragment='paginationbar'>
03     <div class='pagination pagination-centered'>
04         <ul>
05             <li th:class='${page.firstPage}? 'disabled' : ''>
06                 <span th:if='${page.firstPage}'>- First</span>
07                 <a th:if='${not page.firstPage}' th:href='${@page.url}
08 (page.page=1,page.size=${page.size})'>- First</a>
09             </li>
10             <li th:class='${page.hasPreviousPage}? '' : 'disabled'>
11                 <span th:if='${not page.hasPreviousPage}'><<</span>
12                 <a th:if='${page.hasPreviousPage}' th:href='${@page.url}(page.page=${page.number-
13 1},page.size=${page.size})' title='Go to previous page'><<</a>
14             </li>
15             <li th:each='item : ${page.items}' th:class='${item.current}? 'active' : ''>
16                 <span th:if='${item.current}' th:text='${item.number}'>1</span>
17                 <a th:if='${not item.current}' th:href='${@page.url}
18 (page.page=${item.number},page.size=${page.size})'><span th:text='${item.number}'>1</span></a>
19             </li>
20             <li th:class='${page.hasNextPage}? '' : 'disabled'>
21                 <span th:if='${not page.hasNextPage}'>>></span>
22                 <a th:if='${page.hasNextPage}' th:href='${@page.url}
23 (page.page=${page.number+1},page.size=${page.size})' title='Go to next page'>>></a>
24             </li>
25             <li th:class='${page.lastPage}? 'disabled' : ''>
26                 <span th:if='${page.lastPage}'>Last -></span>
27                 <a th:if='${not page.lastPage}' th:href='${@page.url}
28 (page.page=${page.totalPages},page.size=${page.size})'>Last -></a>
29             </li>
30         </ul>
31     </div>
32 </div>

```

Spring Configuration Change

The last step is to put them all together. Fortunately I did some research before I updated my code. There is a very good blog post [Pagination with Spring MVC, Spring Data and Java Config](#) by Doug Haber. In his blog, Doug mentioned several gotchas, especially the `Pageable` parameter needs some configuration magic:

In order for Spring to know how to convert the parameter to a `Pageable` object you need to configure a `HandlerMethodArgumentResolver`. Spring Data provides a `PageableArgumentResolver` but it uses the old `ArgumentResolver` interface instead of the new (Spring 3.1) `HandlerMethodArgumentResolver` interface. The XML config can handle this discrepancy for us, but since we're using Java Config we have to do it a little more manually. Luckily this can be easily resolved if you know the right magic incantation...

Doug Haber

With Doug's help, I added this argument resolver to my `WebConfig` class:

```

01 @Configuration
02 @EnableWebMvc
03 @ComponentScan(basePackages = 'com.jiwhiz.blog.web')
04 public class WebConfig extends WebMvcConfigurerAdapter {
05     ...
06     @Override
07     public void addArgumentResolvers(List<HandlerMethodArgumentResolver> argumentResolvers) {
08         PageableArgumentResolver resolver = new PageableArgumentResolver();
09         resolver.setFallbackPagable(new PageRequest(1, 5));
10         argumentResolvers.add(new ServletWebArgumentResolverAdapter(resolver));
11     }
12     ...
13 }

```

After all those changes, my [Blog](#) list will have pagination bar at the top and bottom of the page, and it always have at most 5 page numbers, with current number at the middle and disabled. The pagination bar also has *First* and *Previous* links at the beginning, *Next* and *Last* links at the end. I also used it in my admin pages, for user list and comment list, and it works very well.

Reference: [Implement Bootstrap Pagination With Spring Data And Thymeleaf](#) from our JCG partner Yuan Ji at the Jiwhiz blog.

You might also like:

- [Spring Data JPA and pagination](#)
- [Spring & JSF integration: Pagination](#)
- [REST Pagination in Spring](#)
- [The Persistence Layer with Spring Data JPA](#)
- [Spring Data JDBC generic DAO implementation – most lightweight ORM ever](#)

Share and enjoy!



DISQUS

DISQUS seems to be taking longer than usual. [Reload?](#)

ERROR

The requested URL could not be retrieved

The following error was encountered while trying to retrieve the URL: <http://disqus.com/embed/comments/>

Knowledge Base

[Examples](#)
[Resources](#)
[Software](#)
[Tutorials](#)

Partners

[Mkyong](#)

The Code Geeks Network

[Java Code Geeks](#)
[.NET Code Geeks](#)
[Web Code Geeks](#)

Hall Of Fame

["Android Full Application Tutorial" series](#)
[GWT 2 Spring 3 JPA 2 Hibernate 3.5 Tutorial](#)
[Android Game Development Tutorials](#)
[Android Google Maps Tutorial](#)
[Android Location Based Services Application – GPS location](#)
[Funny Source Code Comments](#)
[Java Best Practices – Vector vs ArrayList vs HashSet](#)
[Android JSON Parsing with Gson Tutorial](#)
[Android Quick Preferences Tutorial](#)

About Java Code Geeks

JCGs (Java Code Geeks) is an independent online community creating the ultimate Java to Java developers resource. I am a technical architect, technical team lead (senior developer), and a junior developers alike. JCGs serve the Java, SOA, Agile communities with daily news written by domain experts, announcements, code snippets and open source projects.

License



This work by [Java Code Geeks](#) is licensed under a [Creative Commons Attribution-ShareAlike 3.0 Unported License](#).

© 2010-2012 Java Code Geeks. Licensed under a Creative Commons Attribution-ShareAlike 3.0 Unported License.

All trademarks and registered trademarks appearing on Java Code Geeks are the property of their respective owners.

Java is a trademark or registered trademark of Oracle Corporation in the United States and other countries.

Java Code Geeks is not connected to Oracle Corporation and is not sponsored by Oracle Corporation.